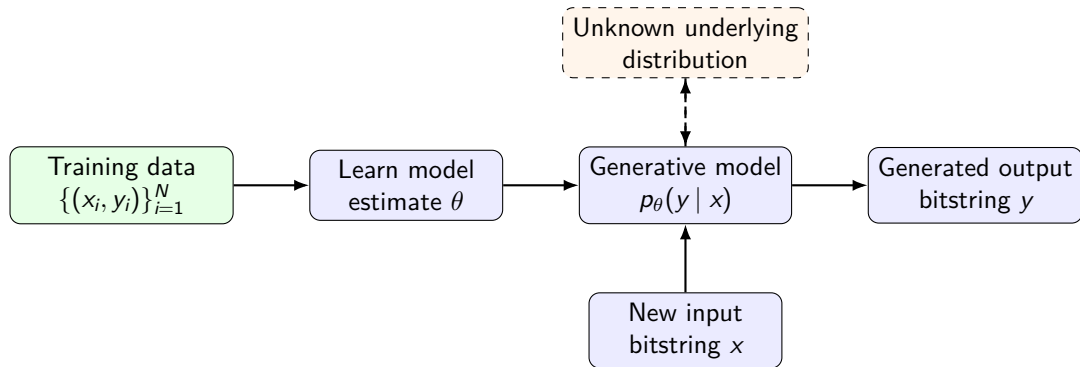


MNIST Training with DQNN/IDQNN

Quantum Machine Learning

April 16, 2026

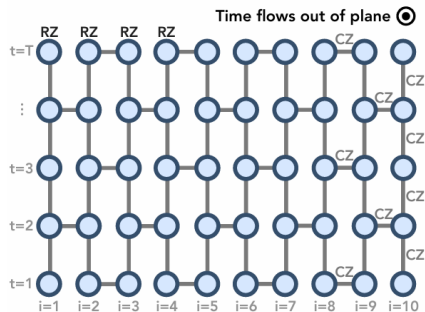
Generate bitstring with DQNN/IDQNN



Learn a conditional generator from training pairs and sample new y for a given x .

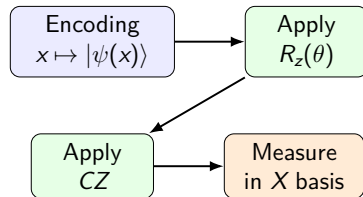
Models: DQNN(Deep Quantum Neural Network) and shallow IDQNN(Instantaneous Deep Quantum Neural Network). $P_{IDQNN}(y | x, \theta) = P_{DQNN}(y | x, \theta)$.

IDQNN Definition and Architecture



2D Shallow QNN

2D shallow QNN architecture used by IDQNN.



$$|\psi_{\text{out}}\rangle = (\prod CZ) (\prod R_z(\theta)) |\psi(x)\rangle, \quad y \sim p_{\theta}(y | x)$$

Input-state encoding rule

Qubit i is initialized from $\{|0\rangle, |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\}$ according to bit x_i : $0 \mapsto |+\rangle$, $1 \mapsto |0\rangle$.

Learning Algorithm: parameters Estimation

Learning stratage

- For each parameter at position (i, j) , filter samples satisfying:

$$x_{i,j} = 0, \quad x_{\mathcal{N}(i,j)} = 1,$$

where $\mathcal{N}(i, j)$ denotes positions connected via CZ gates.

- Compute the parameter estimate from the filtered samples.

$$\hat{\theta}_{i,j} = \arccos \left(1 - \frac{2}{N_{\text{sp}}} \sum_{t=1}^{N_{\text{sp}}} y_{(t)}^{(i,j)} \right).$$

Dataset generation

Using ideal parameters θ^* and IDQNN circuit, generate dataset:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N_{\text{sample}}}.$$

Quantum models can learn underlying distributions over $\mathcal{D}$ which classical models struggle with.

New Question: MNIST Training Performance

Question

How well do DQNN/IDQNN train on a practical handwritten-digit task?

- MNIST training set: 60,000 samples; test set: 10,000 samples.
- Original image size: $28 \times 28 = 784$ grayscale pixels, normalized to $[0, 1]$.
- We construct a learning dataset $\mathcal{D}_{\text{MNIST}} = \{(x_i, y_i)\}_{i=1}^{60000}$.

Key practical point

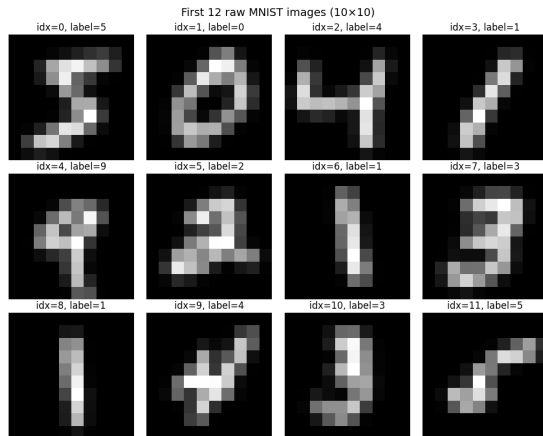
DQNN and IDQNN generate the same distribution, but runtime differs significantly in practice.

- DQNN uses only m qubits; IDQNN uses $n_1 \times m$ qubits in the shallow representation.
- Measured sampling runtime in our implementation:

Model	input $\times$ size	Sampling time (single run)
DQNN	10×10 ($n = 100$)	1.8 s
IDQNN	5×5 ($n = 25$)	44.019 s

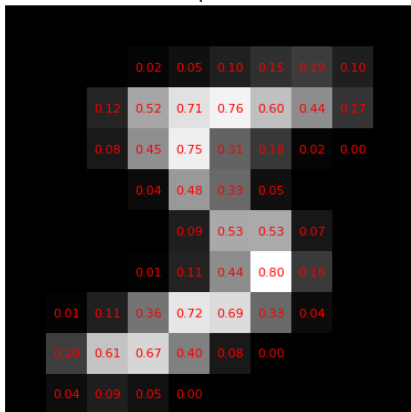
Preprocessing: Resize to 10×10 for Input x

- We reshape each MNIST image to a 10×10 grayscale image.
- The resulting 100-pixel vector is mapped to the input bitstring x .



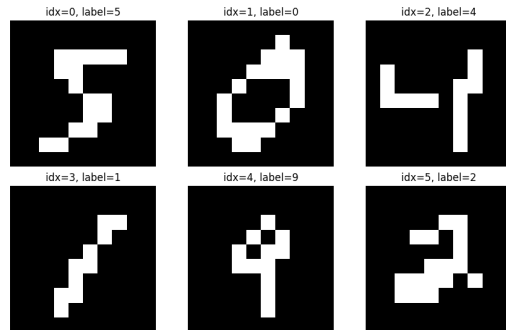
Binarization and Threshold Choice

Non-zero pixel values



Data Binarization:

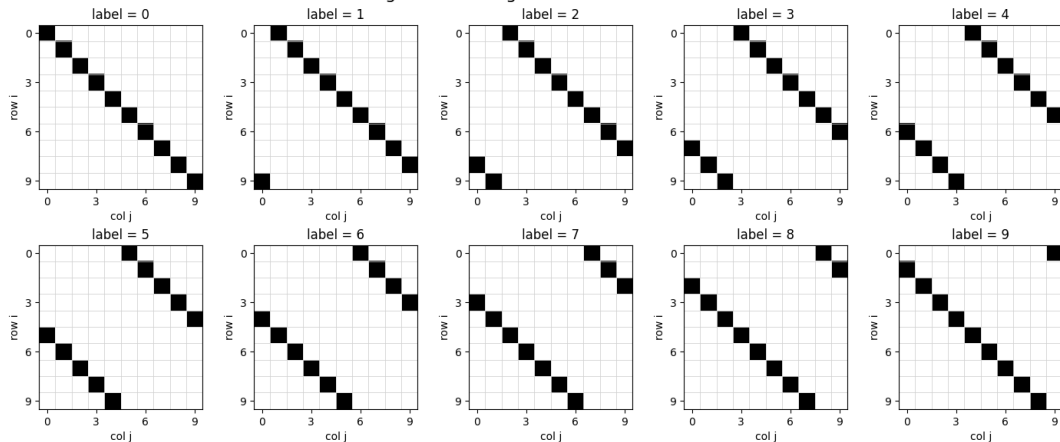
First 6 binarized(0.4) MNIST images (10×10)



Gain the train input data $\{x_i\}$.

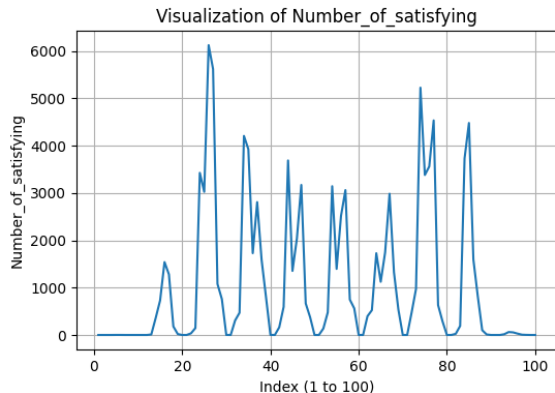
Label-Bitstring Encoding for y (0–9)

Diagonal encoding matrices for labels 0-9



Encoding pattern for digit labels 0–9. Gain the datasets $\{x_i, y_i\}$, both x and y are 100-bit strings.

Filter-Sample Statistics for Parameter Estimation



Valid Sample Statistics

- We first count valid filter samples for each parameter position.
- Skipped bits ($N_{sp} = 0$): 21.
- Indices (0-based):
[0, 1, 2, 3, ..., 29, ..., 50, ..., 99].

Parameters with no valid samples are set to 0.

Generation on Test Set with Learned Parameters

- After estimating θ , we generate y on test inputs.
- Example: for test index 99, we sampled 1000 generated outputs and averaged the generated bits.
- The averaged generated pattern is compared with the true encoded label.

